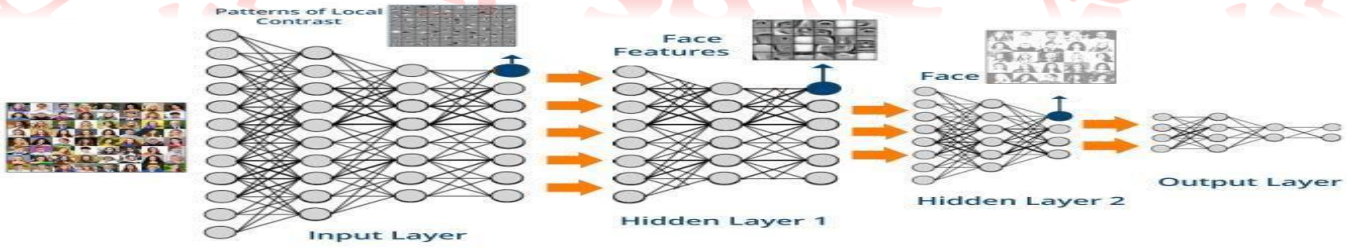
机器学习和深度学习



手写识别体案例分享

陈会

目录
Contents

1 Scikit-learn概述

2 Tensorflow

3 PaddlePaddle

4 PyTorch

5 Keras

1 手写识别体数据库

MNIST数据集

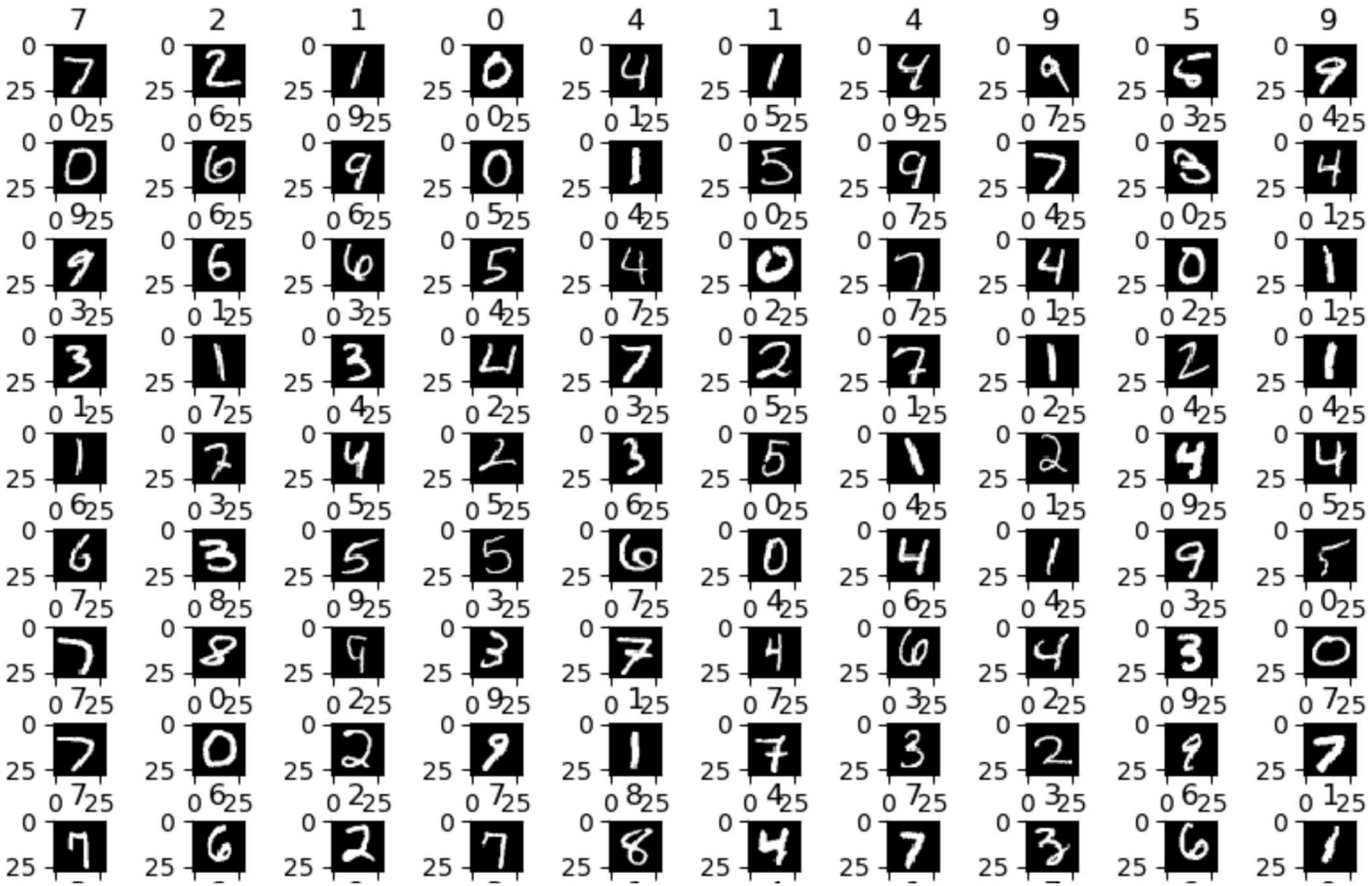
MNIST数据集由Yann LeCun搜集，是一个大型的手写体数字数据库，通常用于训练各种图像处理系统，也被广泛用于机器学习领域的训练和测试。

数据集是有250人手写数字组成，一半是高中生，一半是美国人口普查局。MNIST数据集共有训练数据60000项、测试数据10000项。每张图像的大小为28*28（像素），每张图像都为灰度图像，位深度为8（灰度图像是0-255）。

手动下载地址：
http://yann.lecun.com/exdb/mnist/

train-images-idx3-ubyte.gz:	training set images (9912422 bytes)
train-labels-idx1-ubyte.gz:	training set labels (28881 bytes)
t10k-images-idx3-ubyte.gz:	test set images (1648877 bytes)
t10k-labels-idx1-ubyte.gz:	test set labels (4542 bytes)

MNIST数据集


7 2 1 0 4 1 4 9 5 9
0 0 6 9 0 1 5 9 7 3 4
9 6 6 5 4 0 7 4 0 1
3 1 3 4 7 2 7 1 2 1
1 7 4 2 3 5 1 2 4 4
6 3 5 5 6 0 4 1 9 5
7 8 9 3 7 4 6 4 3 0
7 0 2 9 1 7 3 2 9 7
7 6 2 7 8 4 7 3 6 1

1 机器学习库Scikit-learn

1.Scikit-learn概述

Scikit-learn是基于NumPy、 SciPy和 Matplotlib的开源Python机器学习包,它封装了一系列数据预处理、机器学习算法、模型选择等工具,是数据分析师首选的机器学习工具包。

自2007年发布以来， scikit-learn已经成为Python重要的机器学习库了，scikit-learn简称sklearn，支持包括分类，回归，降维和聚类四大机器学习算法。还包括了特征提取，数据处理和模型评估三大模块。

scikit-learn: machine learning in Python — scikit-learn 1.5.2 documentation

KNeighborsRegressor — scikit-learn 1.5.2 documentation

2.Scikit-learn主要用法

符号标记

`X_train` | 训练数据.

`X_test` | 测试数据.

`X` | 完整数据.

`y_train` | 训练集标签.

`y_test` | 测试集标签.

`y` | 数据标签.

2.Scikit-learn主要用法

基本建模流程

导入工具包

```
from sklearn import datasets, preprocessing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

2.Scikit-learn主要用法

加载数据

✓Scikit-learn支持以NumPy的arrays对象、 Pandas对象、 SciPy的稀疏 矩阵及其他可转换为数值型arrays的数据结构作为其输入， 前提是数据必须是数值型的

✓sklearn.datasets模块提供了一系列加载和获取著名数据集如鸢尾花、 波士顿房价、 Olivetti人脸、 MNIST数据集等的工具， 也包括了一些toy data如S型数据等的生成工具

```
from sklearn.datasets import load_iris
iris = load_iris ()
X = iris.data
y = iris.target
```

2.Scikit-learn主要用法

数据划分

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test =  train_test_split (X, y,
random_state=12, stratify=y,  test_size=0.3)
```

将完整数据集的70%作为训练集， 30%作为测试集， 并使得测试集和训练集中各类别数据的比例与原始数据集比例一致（stratify分层策略），另外 可通过设置shuffle=True 提前打乱数据。random_state参数的值，是设置的随机种子，train_test_split函数根据不同的随机种子，将数据集划分为训练集和测试集。

2.Scikit-learn主要用法

数据预处理

使用Scikit-learn进行数据标准化

```
from sklearn.preprocessing import StandardScaler
```

构建转换器实例

```
scaler = StandardScaler ()
```

拟合及转换

```
scaler.fit_transform (X_train)
```

Z-Score标准化

$$x^* = \frac{x - \mu}{\sigma}$$

$$\sigma^2 = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)} - \mu)^2$$

$$\mu = \frac{1}{m}\sum_{i=1}^{m}x^{(i)}$$

处理后的数据均值为0，方差为1

2.Scikit-learn主要用法

数据预处理

使用Scikit-learn进行数据变换

最小最大标准化	MinMaxScaler
One-Hot编码	OneHotEncoder
归一化	Normalizer
二值化（单个特征转换）	Binarizer
标签编码	LabelEncoder
缺失值填补	Imputer
多项式特征生成	PolynomialFeatures

归一化（最大 - 最小规范化）

$$x^* = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

将数据映射到[0,1]区间

2.Scikit-learn主要用法

特征选择

```
from sklearn import feature_selection as fs
fs.SelectKBest (score_func, k)
```

过滤式（Filter），保留得分排名前k的特征（top k方式）

```
fs.RFECV (estimator,  scoring="r2")
```

封装式（Wrapper），结合交叉验证的递归特征消除法， 自动选择最优特征个数

```
fs.SelectFromModel (estimator)
```

嵌入式（Embedded），从 模型中自动选择特征，任何具有coef_或者feature_importances_的 基模型都可以作为estimator参数传入

2.Scikit-learn主要用法

监督学习算法-回归

```
from sklearn.linear_model import LinearRegression
```

构建模型实例

```
lr = LinearRegression (normalize=True)
```

训练模型

```
lr.fit (X_train, y_train)
```

作出预测

```
y_pred = lr.predict (X_test)
```

LASSO `linear_model.Lasso`

Ridge `linear_model.Ridge`

ElasticNet `linear_model.ElasticNet`

回归树 `tree.DecisionTreeRegressor`

2.Scikit-learn主要用法

监督学习算法-分类

```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier (max_depth=5)
clf.fit (X_train, y_train)
y_pred = clf.predict (X_test)
y_prob = clf.predict_proba (X_test)
```

使用决策树分类算法解决二分类问题， y_prob 为每个样本预测为“0”和 “1”类的概率

1.Scikit-learn概述

监督学习算法-分类

逻辑回归	`linear_model.LogisticRegression`
支持向量机	`svm.SVC`
朴素贝叶斯	`naive_bayes.GaussianNB`
K近邻	`neighbors.NearestNeighbors`

2.Scikit-learn主要用法

监督学习算法-集成学习

sklearn.ensemble模块包含了一系列基于集成思想的分类、回归和离群值检测方法.

```
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier (n_estimators=20)
clf.fit (X_train, y_train)
y_pred = clf.predict (X_test)
y_prob = clf.predict_proba (X_test)
```

AdaBoost	ensemble.AdaBoostClassifier ensemble.AdaBoostRegressor
基于梯度提升	ensemble.GradientBoostingClassifier ensemble.GradientBoostingRegressor

2.Scikit-learn主要用法

无监督学习算法

sklearn.cluster模块包含了一系列无监督聚类算法.

```
from sklearn.cluster import KMeans
```

构建聚类实例

```
kmeans = KMeans (n_clusters=3, random_state=0)
```

拟合

```
kmeans.fit (X_train)
```

预测

```
kmeans.predict (X_test)
```

2.Scikit-learn主要用法

无监督学习算法-降维

sklearn.decomposition 模块包含了一系列无监督降维算法

```
from sklearn.decomposition import PCA
```

导入PCA库，设置主成分数量为3， n_components代表主成分数量

```
pca = PCA(n_components=3)
```

训练模型

```
pca.fit (X)
```

投影后各个特征维度的方差比例(这里是三个主成分)

```
print (pca.explained_variance_ratio_ )
```

投影后的特征维度的方差

```
print (pca.explained_variance_ )
```

2.Scikit-learn主要用法

无监督学习算法-聚类

DBSCAN	`cluster.DBSCAN`
层次聚类	`cluster.AgglomerativeClustering`
谱聚类	`cluster.SpectralClustering`

2.Scikit-learn主要用法

评价指标

sklearn.metrics模块包含了一系列用于评价模型的评分函数、损失函数以及成对数据的距离度量函数.

```
from sklearn.metrics import accuracy_score
accuracy_score (y_true, y_pred)
```

对于测试集而言， y_test即是y_true，大部分函数都必须包含真实值y_true和预测值y_pred.

2.Scikit-learn主要用法

评价指标

针对一个二分类问题，即将实例分成正类（positive）或负类（negative），在实际分类中会出现以下四种情况：

（1）若一个实例是正类，并且被预测为正类，即为真正类TP(True Positive)
（2）若一个实例是正类，但是被预测为负类，即为假负类FN(False Negative)
（3）若一个实例是负类，但是被预测为正类，即为假正类FP(False Positive)
（4）若一个实例是负类，并且被预测为负类，即为真负类TN(True Negative)

预测标签 / 真实标签	正例	反例
正例	TP（真正类）	FN（假反类）
反例	FP（假正类）	TN（真反类）

知乎 @人工智能

2.Scikit-learn主要用法

评价指标

◆ **准确率(Accuracy)**：预测正确的样本数量占总量的百分比

$$Accuracy = \frac{TP+TN}{TP+FN+FP+TN}$$

准确率有一个**缺点**，就是数据的样本不均衡，这个指标是不能评价模型的性能优劣的。假如一个测试集有正样本99个，负样本1个。模型把所有的样本都预测为正样本，那么模型的Accuracy为99%，看评价指标，模型的效果很好，但实际上模型没有任何预测能力。

◆ **精准率(Precision)**：又称为查准率，是针对预测结果而言的一个评价指标。**在模型预测为正样本的结果中，真正是正样本所占的百分比。**

$$Accuracy = \frac{TP}{TP+FP}$$

2.Scikit-learn主要用法

评价指标

- **召回率(Recall)：**又称为**查全率**，是针对原始样本而言的一个评价指标。**在实际为正样本中，被预测为正样本所占的百分比。**

$$Accuracy = \frac{TP}{TP+FN}$$

- **F1 Score：**针对精准率和召回率都有其自己的缺点；如果阈值较高，那么精准率会高，但是会漏掉很多数据；如果阈值较低，召回率高，但是预测的会很不准确。

$$F1 = \frac{2 \times P \times R}{P+R}$$

2.Scikit-learn主要用法

评价指标

回归模型评价

`metrics.mean_absolute_error()` | 平均绝对误差MAE

`metrics.mean_squared_error()` | 均方误差MSE

`metrics.r2_score()` | 决定系数R^2.

2.Scikit-learn主要用法

评价指标

分类模型评价

`metrics.accuracy_score()` | 正确率

`metrics.precision_score()` | 各类精确率

`metrics.f1_score()` | F1 值

`metrics.log_loss()` | 对数损失或交叉熵损失

`metrics.confusion_matrix` | 混淆矩阵

`metrics.classification_report` | 含多种评价的分类报告

2.Scikit-learn主要用法

评价指标

分类模型评价

metrics.accuracy_score () | 正 确 率 .

metrics.precision_score () | 各类精确率.

metrics.f1_score() | F1 值 .

metrics.log_loss () | 对数损失或交叉熵损失.

metrics.confusion_matrix | 混淆矩阵.

metrics.classification_report | 含多种评价的分类报告.

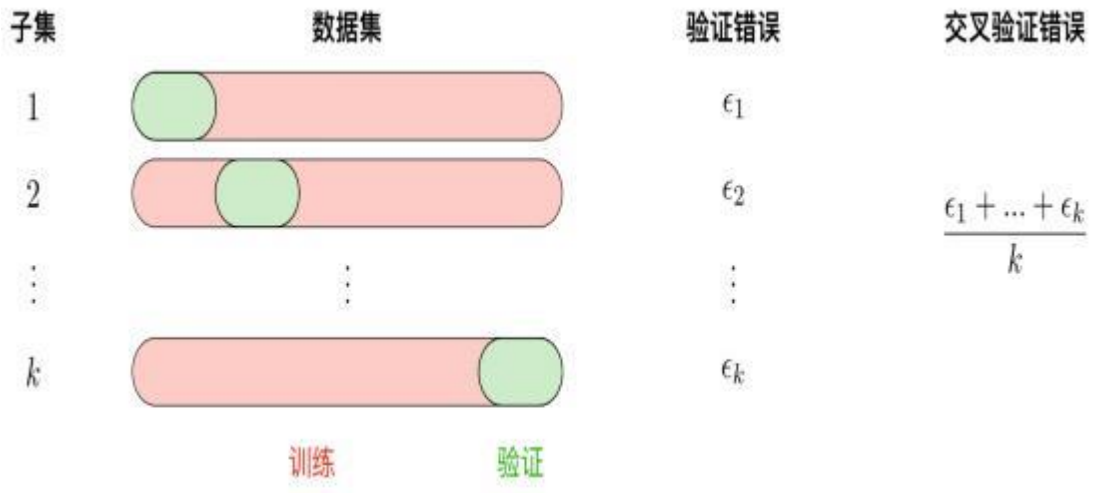
2.Scikit-learn主要用法

交叉验证及超参数调优

```
from sklearn.model_selection import cross_val_score
clf = DecisionTreeClassifier (max_depth=5)
scores = cross_val_score (clf, X_train, y_train,
            cv=5, scoring='f1_weighted')
```

使用5折交叉验证对决策树模型进行评估，使用的评分函数为F1值

sklearn提供了部分带交叉验证功能的模型类如LassoCV、 LogisticRegressionCV等，这些类包含cv参数



2.Scikit-learn主要用法

交叉验证及超参数调优

超参数调优— 网格搜索

```
from sklearn.model_selection import GridSearchCV
from sklearn import svm
svc = svm.SVC ()
params = {'kernel':['linear', 'rbf'], 'C':[1, 10]}  grid_search =
GridSearchCV (svc, params, cv=5)  grid_search.fit (X_train, y_train)
grid_search.best_params_
```

在参数网格上进行穷举搜索，方法简单但是**搜索速度慢**（超参数较多时），且不容易找到参数空间中的局部最优

2.Scikit-learn主要用法

交叉验证及超参数调优

超参数调优— 随机搜索

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint
svc = svm.SVC ()
param_dist = {'kernel':['linear', 'rbf'], 'C':randint (1, 20)}
random_search = RandomizedSearchCV (svc, param_dist, n_iter=10)
random_search.fit (X_train, y_train)
random_search.best_params_
```

在参数子空间中进行随机搜索，选取空间中的100个点进行建模（可从scipy.stats常见分布如正态分布norm、均匀分布uniform中随机采样得到），时间耗费较少，更容易找到局部最优

2 Tensorflow

Tensorflow的起源

- 2015年11月6日，Google公司宣布推出全新的机器学习开源工具Tensorflow。
- Tensorflow最初由Google Brain团队开发，基于Google开发的深度学习基础架构DistBelief构建起来的。
- 官网：https://tensorflow.google.cn/
- Github：https://github.com/tensorflow/tensorflow

TensorFlow

Tensorflow的组成

- TensorFlow：一个采用计算图的形式表述数值计算的编程框架。
- 节点：计算图中每一个节点表示一次数学计算。
- 边：计算图中的每一条边表示计算之间的依赖关系。
- 张量（Tensor）：计算图的基本数据结构，可以理解为多维数据。
- 流（Flow）：张量之间通过计算互相转化的过程。

Tensorflow中的张量

- 是一个物理量，对高维 (维数 ≥ 2) 的物理量进行 “量纲分析” 的一种工具。
- 简单的可以理解为：
  - 一维数组称为矢量
  - 二维数组为二阶张量
  - 三维数组为三阶张量
  - …

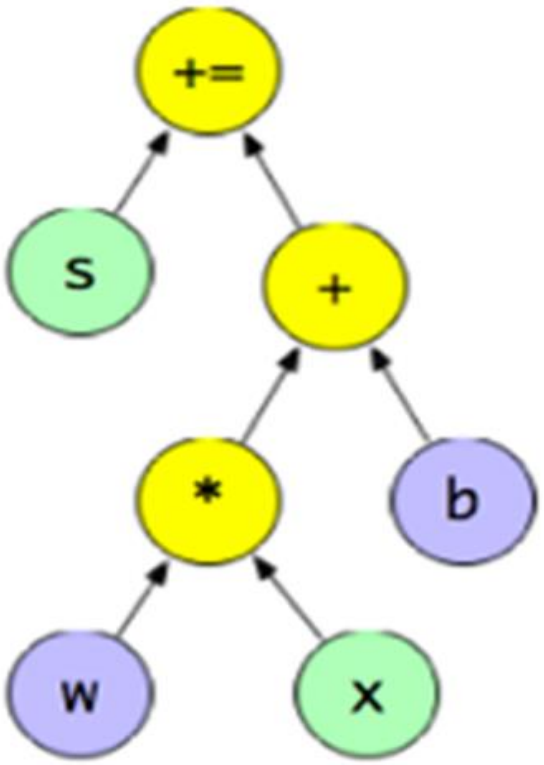
Tensorflow中的计算图

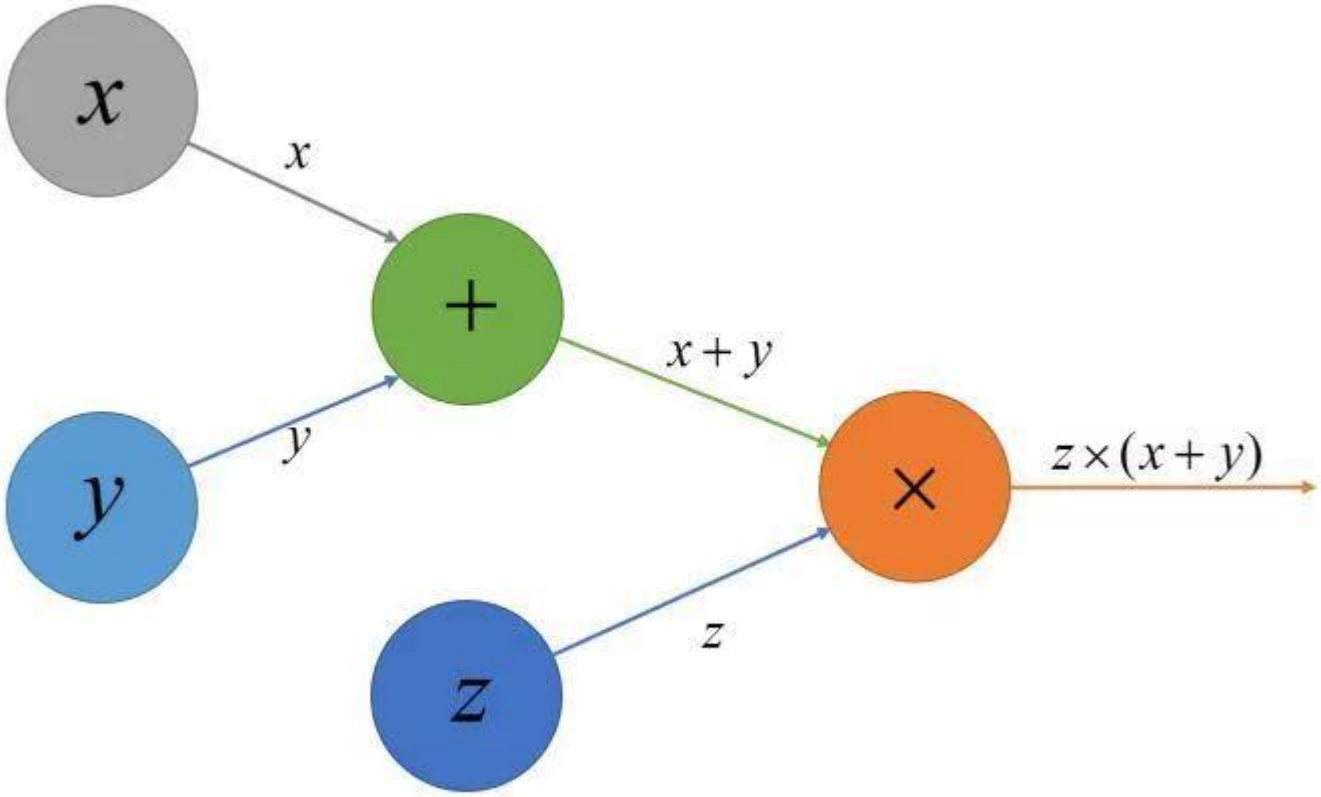
- 计算图：用“节点”（Nodes）和“边”（Edges）的有向图来描述数学计算的图。
- 节点：表示施加的数学操作，但也可以表示数据输入（feed in）的起点/输出（push out）的终点，或者是读取/写入持久变量（persistent variable）的终点。
- 边：表示“节点”之间的输入/输出关系。“边”可以运输“大小可动态调整”的“张量”。

Tensorflow中的计算图

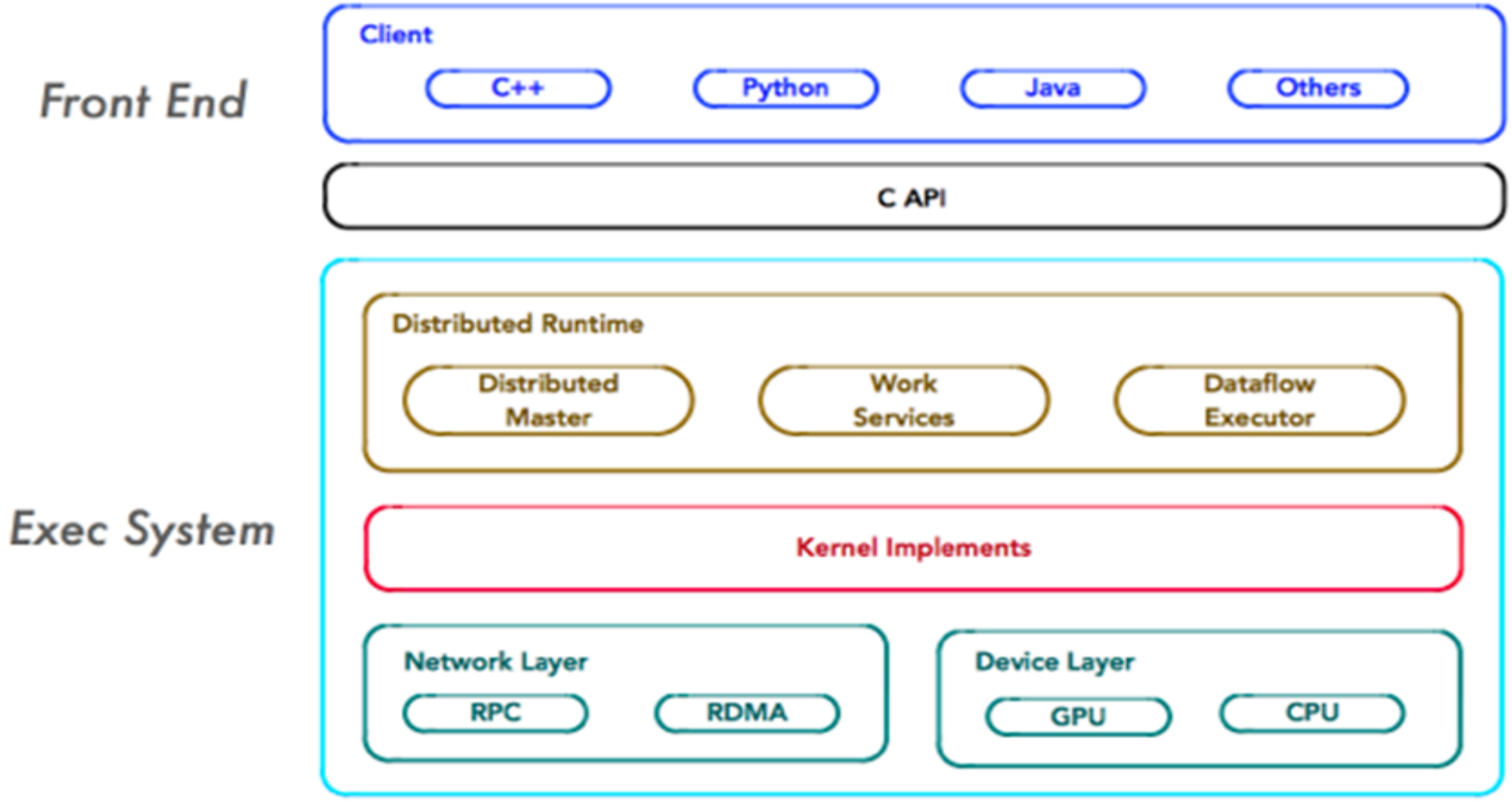
- 节点类型
  - Operation：有一个或者两个输入节点的节点，执行某个操作。
  - Constant：没有输入节点的节点，执行过程中节点的值不变。
  - Variable：没有输入节点的节点，执行过程中节点中的值会变。
  - Placeholder：没有输入节点的节点，执行过程中等待外部输入。

Tensorflow中的计算图





Tensorflow架构



Tensorflow架构

- 前端系统
  - 提供多语言编程环境，提供统一的编程模型来构造计算图。
  - 通过Session的形式，连接TensorFlow后端的「运行时」系统，启动计算图的执行过程。
- 执行系统：提供运行时环境，负责执行计算图。

Tensorflow的使用

- 使用 张量（Tensor） 表示数据。
- 通过变量 （Variable）输入训练数据，维护状态。
- 使用计算图 （Computational graph）来表示计算任务。
- 在会话 （Session）的上下文 （Context） 中执行计算图。

Tensorflow编程示例1：创建计算图

```
import tensorflow as tf

#使用tensorflow在默认的图中创建节点，这个节点是一个变量，命名为“counter”
state = tf.Variable(0, name="counter")

#创建一个常量
one = tf.constant(1)

#对常量与变量进行简单的加法操作，这点需要说明的是： 在TensoorFlow中，
#所有的操作op和变量都视为节点，tf.add() 的意思就是在tf的默认图中添加一个op，这个op是用来做加法操作的。
new_value = tf.add(state,one)

#赋值操作。将new_value的值赋值给update变量
update = tf.assign(state,new_value)

```

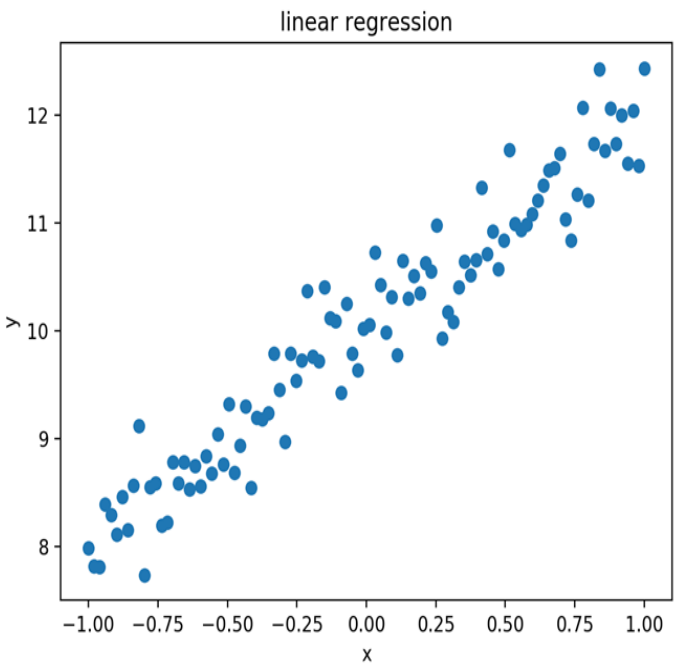
Tensorflow编程示例1：创建计算图

```
#此处用于初始化变量。但是这句话仍然不会立即执行。需要通过sess来将数据流动起来 。如果有Variable，一定需要写这句话
init = tf.initialize_all_variables()

#切记：所有的运算都应该在session中进行，即 with tf.Session() as sess
with tf.Session() as sess:
    #对变量进行初始化，执行（run）init语句
    sess.run(init)
    #循环3次，并且打印输出。
    for i in range(3):
        sess.run(update)
        # 打印变量时也需要用sess.run
        print(sess.run(state))
```

```
1
2
3

Process finished with exit code 0
```

Tensorflow编程示例2：一元线性回归

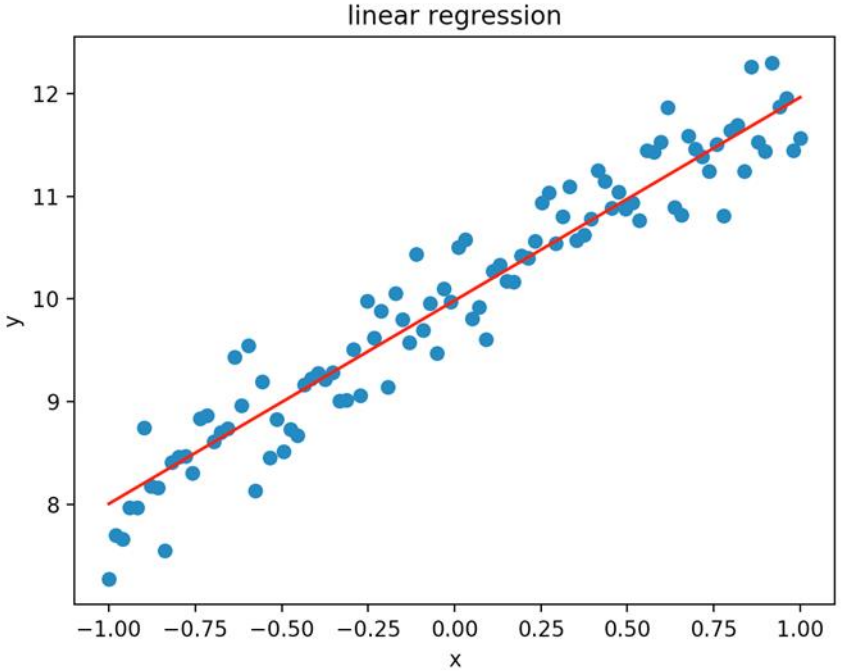
```python
# -*- coding: utf-8 -*-
import tensorflow as tf
import numpy as np

# 散点图数据，理论 w=2, b=10
train_X = np.linspace(-1, 1, 100) #[-1,1)之间的100个数
train_Y = 2 * train_X + \
          np.random.randn(*train_X.shape) * 0.33 + 10

# 构建计算图
X = tf.placeholder(tf.float32, name="x")
Y = tf.placeholder(tf.float32, name="y")
w = tf.Variable(0.0, name="weight")
b = tf.Variable(0.0, name="bias")
loss = tf.square(Y - X * w - b) #损失函数为离差平方和
train_op = tf.train.GradientDescentOptimizer(0.01).minimize(loss) #梯度下降算法
```



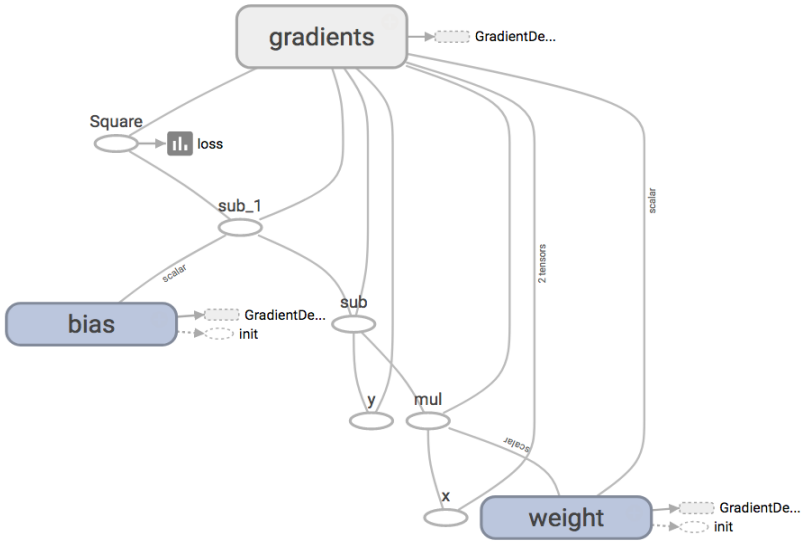
Tensorflow编程示例2：一元线性回归

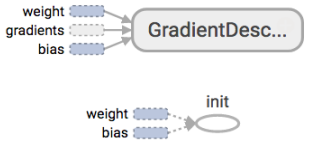
```python
# 运行计算图
with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())

    for i in range(20):#20次迭代
        for (x, y) in zip(train_X, train_Y):
            op_result, w_result, b_result = \
                sess.run([train_op, w, b],feed_dict={X: x, Y: y})
        print("Epoch: {}, w: {}, b: {}".format(i+1, w_result, b_result))
```

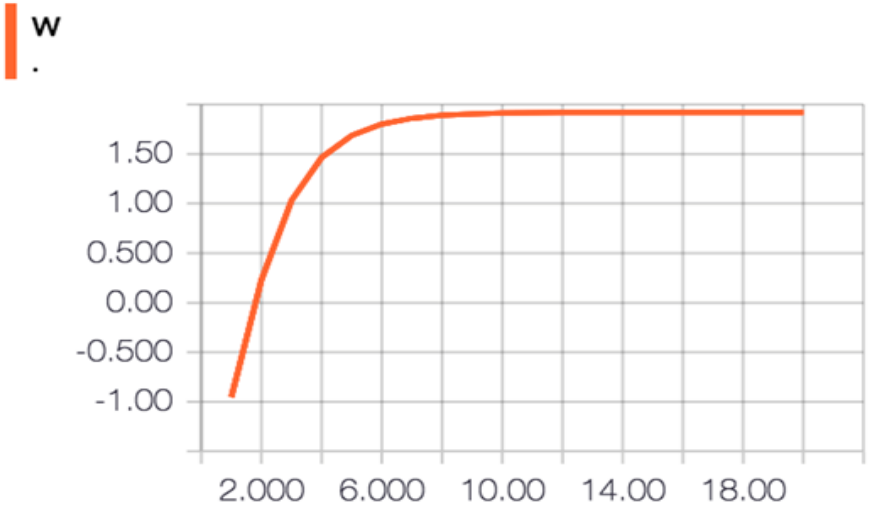
```
Epoch: 14, w: 1.95590674877, b: 10.0173501968
Epoch: 15, w: 1.95616412163, b: 10.0172538757
Epoch: 16, w: 1.95629501343, b: 10.0172052383
Epoch: 17, w: 1.95636260509, b: 10.0171775818
Epoch: 18, w: 1.95639693737, b: 10.017162323
Epoch: 19, w: 1.95641410351, b: 10.0171556473
Epoch: 20, w: 1.95642268658, b: 10.0171527863

Process finished with exit code 0
```

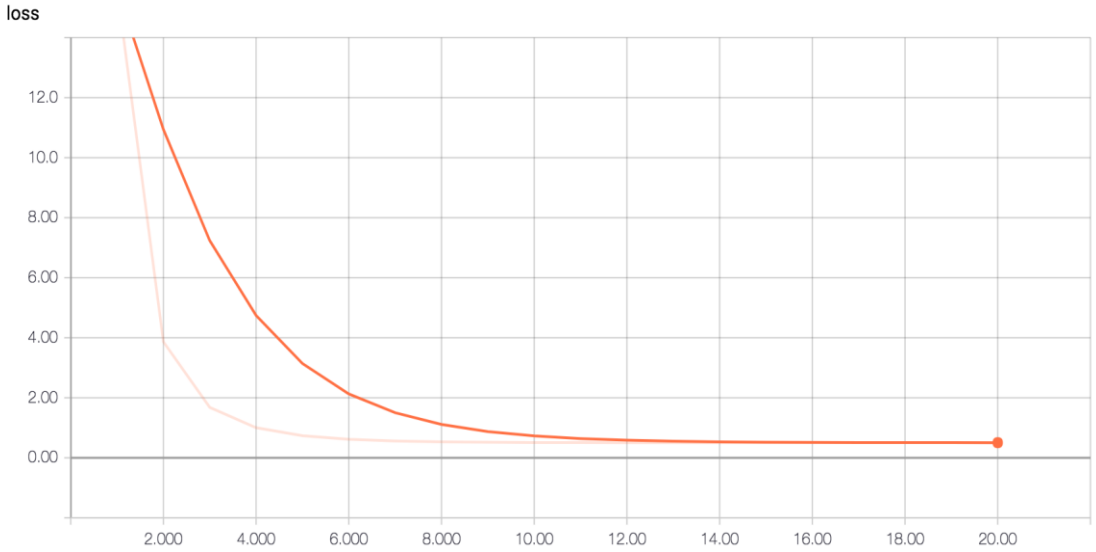


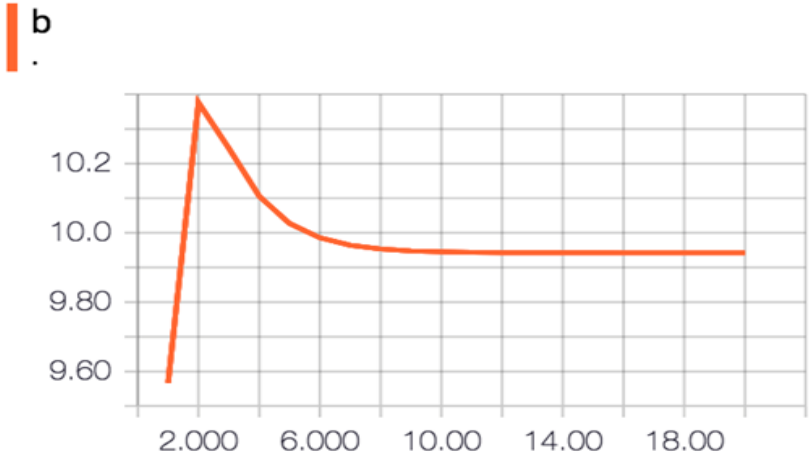
Tensorflow编程示例2：一元线性回归











Tensorflow编程示例3：BP网络手写数字识别

- 导入相关包及数据包

```
import tensorflow as tf
import urllib
from  tensorflow.examples.tutorials.mnist  import  input_data
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image, ImageFilter
mnist = input_data.read_data_sets("MNIST_data/", one_hot = True)
```

```
输出结果:
Extracting MNIST_data/train-images-idx3-ubyte.gz
Extracting MNIST_data/train-labels-idx1-ubyte.gz
Extracting MNIST_data/t10k-images-idx3-ubyte.gz
Extracting MNIST_data/t10k-labels-idx1-ubyte.gz
```

Tensorflow编程示例3：BP网络手写数字识别

- 建立BP网络模型

```
#建立BP神经网络模型
num_classes = 10#数据类型0-9
input_size = 784#28*28
hidden_units_size = 30#层节点数
batch_size = 100#
training_iterations = 50000#迭代次数
# 设置变量
X = tf.placeholder (tf.float32, shape = [None, input_size])
Y = tf.placeholder (tf.float32, shape = [None, num_classes])
W1 = tf.Variable (tf.random_normal ([input_size, hidden_units_size],
                                    stddev = 0.1))#hidden_units_size = 30#正态分布随机数
B1 = tf.Variable (tf.constant (0.1),
                  [hidden_units_size])#常数为1，形状为（1,1）
W2 = tf.Variable (tf.random_normal ([hidden_units_size,
                                     num_classes], stddev = 0.1))#正态分布随机数
B2 = tf.Variable (tf.constant (0.1), [num_classes])
```

Tensorflow编程示例3： BP网络手写数字识别

- 建立BP网络模型

```
# 搭建计算网络 使用 relu 函数作为激励函数 这个函数就是 y = max (0,x) 的一个类似线性函数 拟合程度还是不错的
# 使用交叉熵损失函数 这是分类问题例如： 神经网络 对率回归经常使用的一个损失函数
#第1层神经网络
hidden_opt = tf.matmul (X, W1) + B1#矩阵运算
hidden_opt = tf.nn.relu (hidden_opt)#激活函数
#第2层神经网络
final_opt = tf.matmul (hidden_opt, W2) + B2#矩阵运算
final_opt = tf.nn.relu (final_opt)#激活函数,最终的输出结果
loss = tf.reduce_mean (
    tf.nn.softmax_cross_entropy_with_logits (labels = Y, logits = final_opt))#损失函数,交叉熵方法
opt = tf.train.GradientDescentOptimizer (0.1).minimize (loss)
init = tf.global_variables_initializer ()#全局变量初始化
correct_prediction = tf.equal (tf.argmax (Y, 1), tf.argmax (final_opt, 1))
accuracy = tf.reduce_mean (tf.cast (correct_prediction, 'float'))#将张量转化成float
```

Tensorflow编程示例3：BP网络手写数字识别

- 模型训练及结果

```python
# 进行计算 打印正确率
sess = tf.Session ()#生成能进行TensorFlow计算的类
sess.run (init)
for i in range (training_iterations) :
    batch = mnist.train.next_batch (batch_size)#每次迭代选用的样本数100
    batch_input = batch[0]
    batch_labels = batch[1]
    training_loss = sess.run ([opt, loss], feed_dict = {X: batch_input, Y: batch_labels})
    if (i+1) % 10000 == 0 :
        train_accuracy = accuracy.eval (session = sess, feed_dict = {X: batch_input,Y: batch_labels})
        print ("step : %d, training accuracy = %g " % (i+1, train_accuracy))
```

```
step : 10000, training accuracy = 0.98
step : 20000, training accuracy = 0.98
step : 30000, training accuracy = 1
step : 40000, training accuracy = 1
step : 50000, training accuracy = 1
```

Tensorflow编程示例3：BP网络手写数字识别

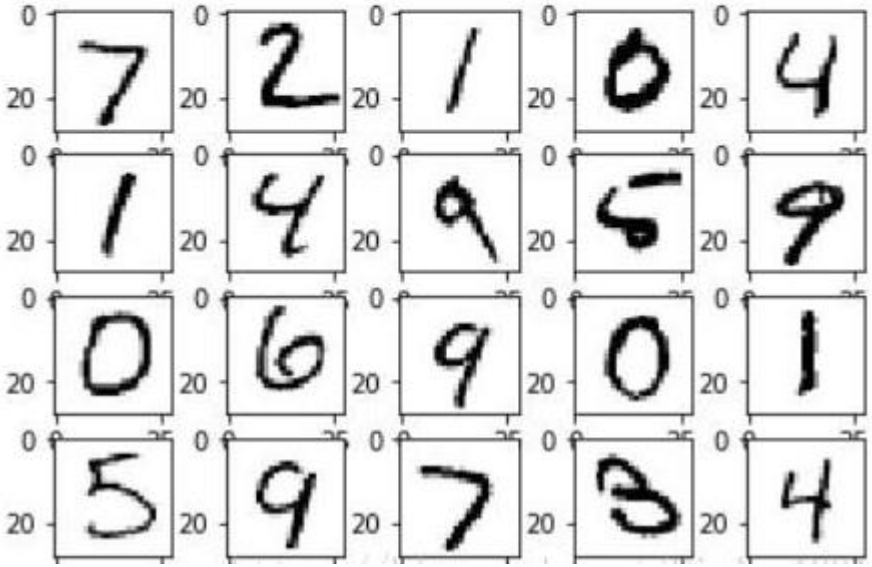
- 测试结果可视化

```
###测试集输出结果可视化
def res_Visual(n):
    #sess=tf.Session()
    #sess.run(tf.global_variables_initializer())
    final_opt_a=tf.argmax (final_opt, 1).eval(session=sess,feed_dict = {X: mnist.test.images,Y: mnist.test.labels})
    fig, ax = plt.subplots(nrows=int(n/5),ncols=5 )
    ax = ax.flatten()
    print('前{}张图片预测结果为: '.format(n))
    for i in range(n):
        print(final_opt_a[i],end=',')
        if int((i+1)%5) ==0:
            print('\t')
        #图片可视化展示
        img = mnist.test.images[i].reshape((28,28))#读取每行数据，格式为Ndarry
        ax[i].imshow(img, cmap='Greys', interpolation='nearest')#可视化
    print('测试集前{}张图片为: '.format(n))
res_Visual(20)
```

Tensorflow编程示例3：BP网络手写数字识别

- 测试结果

```
前20张图片预测结果为：
7,2,1,0,4,
1,4,9,6,9,
0,6,9,0,1,
5,9,7,3,4,
测试集前20张图片为：
```



3 Keras

Keras的起源

- Keras于2015年由Google人工智能研究员Francois Chollet等人提出。
- Keras是一个高层的神经网络API，由纯Python编写而成并使用其他深度学习框架Tensorflow、Theano及CNTK作为后端。

Keras 中文文档地址：

https://keras.io/zh/models/about-keras-models/

较低的使用门槛

- Keras为支持快速实验而生，能够把想法迅速转换为结果。
- Keras应该是深度学习框架中最容易上手的一个，它提供了一致而简洁的API，能供大大减少一般应用下用户的工作量，对于新用户非常友好。

过度的封装

- 严格意义上来说，Keras并不能称为一个深度学习框架，它更像是一个高层的深度学习接口。Keras的缺点也很明显：过度的封装导致丧失了灵活性。
- 由于同时支持三种不同的深度学习框架作为后端，Keras做了层层封装，导致用户在新增操作或是获取底层的数据信息时过于困难。
- 许多Bug隐藏在层层封装之下，导致Keras的程序过于缓慢。

Keras模型

Keras供的模型，其中分为两类：

•Sequential 顺序模型

from keras.models import Sequential

•Model 类模型

from keras.models import Model

Keras Sequential 顺序模型

Sequential 模型结构： 层（layers）的线性堆栈。简单来说，它是一个简单的线性结构，没有多余分支，是多个网络层的堆叠。

```
model = Sequential([ Dense(32, input_shape=(784,)),
                     Activation('relu’),
                     Dense(10),
                     Activation('softmax'), ])
```

- Dense是一个全连接层，它的激活函数默认为是linear线性函数
- 激活函数可以通过 单独的激活层 实现，也可以通过 构建层时传递activation实现

Keras Sequential 顺序模型

Sequential 模型结构： 层（layers）的线性堆栈。简单来说，它是一个简单的线性结构，没有多余分支，是多个网络层的线性堆叠。

```
model = Sequential([ Dense(32, input_shape=(784,)),
                     Activation('relu’),
                     Dense(10),
                     Activation('softmax'), ])
```

- Dense是一个全连接层，它的激活函数默认为是linear线性函数
- input_shape输入数据的尺寸
- 激活函数可以通过 单独的激活层 实现，也可以通过 构建层时传递activation实现

Keras Sequential 顺序模型

编译

```
model.compile(optimizer='rmsprop’,
              loss='categorical_crossentropy’,
              metrics=['accuracy'])
```

•优化器（opyimizer）,可以是优化器的字符串标识符，也可以是Optimizer类的实

•损失函数（loss function），模型要将其最小化，可以通过字符串标识符指定，可以通过目标函数

•指定指标（metricts）列表， 对于任何分类问题，需要将其设置为metrics = [‘accuracy’]。 度量可以是现有度量的字符串标识符或自定义度量函数。

Keras Sequential 顺序模型

Sequential 模型训练

```
model.fit(data, labels, epochs=10, batch_size=32)
```

•x Numpy训练数据数组（如果模型有单个输入），或Numpy数组列表（如果模型有多个输入）。

•y 与x相似，只不过y代表的是目标标签（target label）。可以是：Numpy目标（标签）数据数组（如果模型具有单个输出）或Numpy数组列表（如果模型具有多个输出）或 输入图层的名称 或None.

•batch_size Integer 或 None，代表每个梯度更新的样本数，默认值为32.

•epochs Integer，模型的训练的 时期数，每个epoch是对x，y的整个迭代。

4 CIFAR-10数据库

CIFAR-10数据集

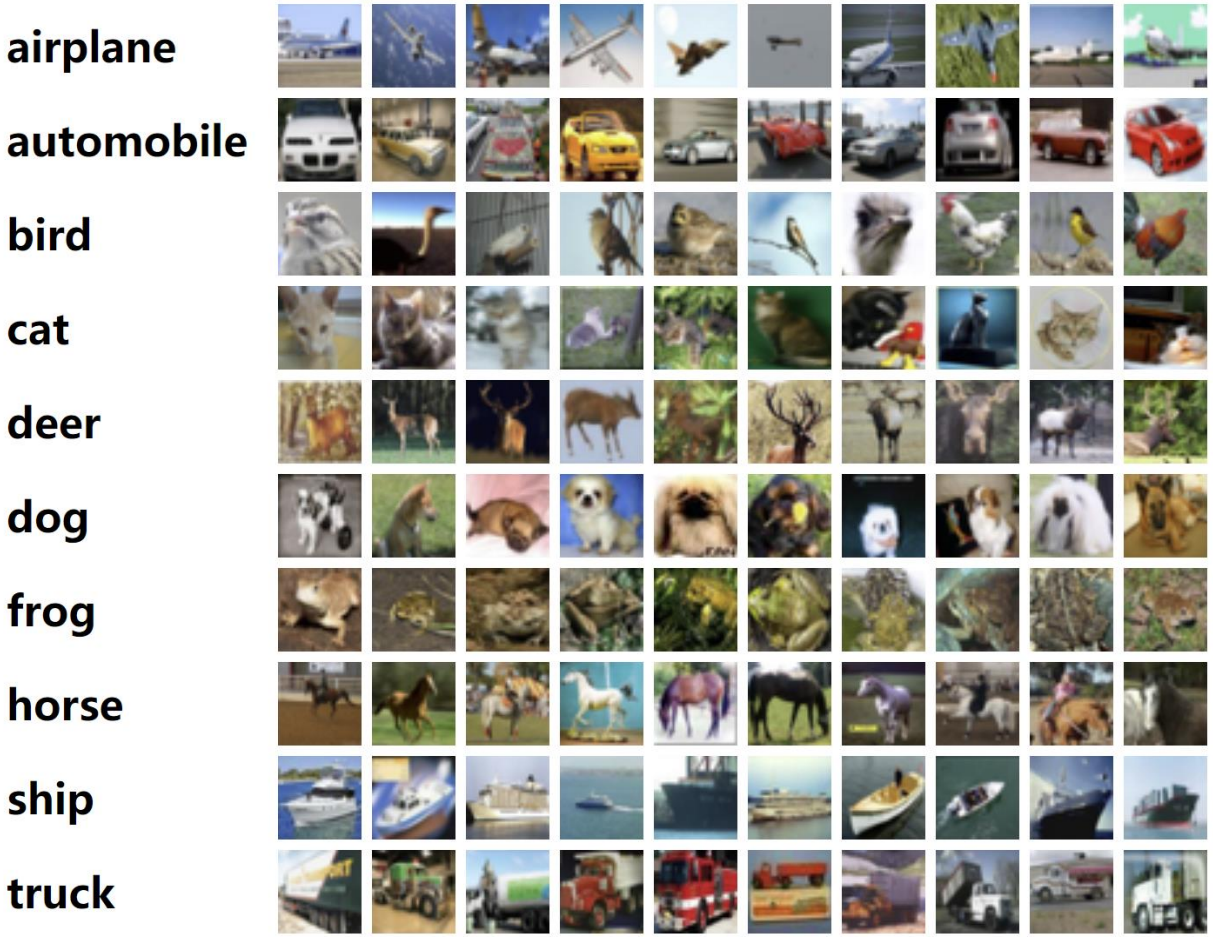
CIFAR-10是一个更接近普适物体的彩色图像数据集。CIFAR-10 是由Hinton 的学生Alex Krizhevsky 和Ilya Sutskever 整理的一个用于识别普适物体的小型数据集。一共包含10 个类别的RGB 彩色图片：飞机（airplane）、汽车（automobile）、鸟类（bird）、猫（cat）、鹿（deer）、狗（dog）、蛙类（frog）、马（horse）、船（ship）和卡车（truck）。

每个图片的尺寸为32 × 32，每个类别有6000个图像，数据集中一共有50000 张训练图片和10000 张测试图片。

手动下载地址：
http://www.cs.toronto.edu/~kriz/cifar.html

CIFAR-10数据集

Here are the classes in the dataset, as well as 10 random images from each:



CIFAR10数据集结构组成可分为这四个部分：

·train_x:(50000, 32, 32, 3)——训练样本

·train_y:(50000, 1)——训练样本标签

·test_x:(10000, 32, 32, 3)——测试样本

·test_y:(10000, 1)——测试样本标签

CIFAR-10数据集卷积神经网络

```
model = Sequential()
#卷积神经网络
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3), padding='same'))
```

添加一个卷积层（Conv2D），该层由32个过滤器，每个过滤器大小为3*3，使用ReLU激活函数（), activation=‘relu‘），输入的形状为input_shape=(32, 32, 3)

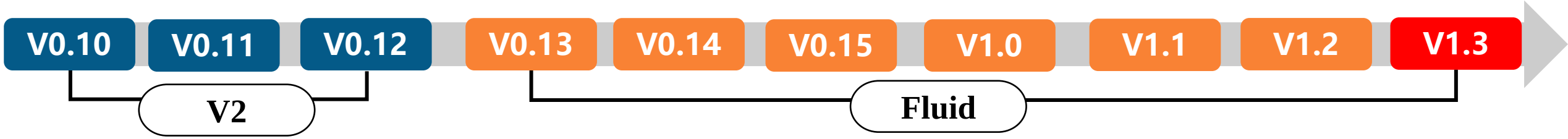
5 PaddlePaddle

PaddlePaddle的发展历程

PaddlePaddle是百度提供的**开源深度学习框架**，致力于为开发者和企业提供最好的深度学习研发体验，框架本身具有易学、易用、安全、高效四大特性，是最适合中国开发者和企业的深度学习工具。

网址：http://www.paddlepaddle.org/zh

- PaddlePaddle前身是2013年百度自主研发的深度学习平台，一直为百度内部工程师使用。
- 2016年9月1日百度世界大会上，百度首席科学家Andrew Ng（吴恩达）首次宣布将百度深度学习平台对外开放，命名PaddlePaddle。
- 截止2017年11月 PaddlePaddle 同其他框架比，前13个月的活跃度是最高。



PaddlePaddle一直在持续更新版本！

PaddlePaddle的功能特点与优势

功能特点

•教程文档友好，提供jupyter文档

开源

•开源，代码在github上公开

• 社区活跃，合作伙伴支持，有很多的贡献者

速度快、方便

• 底层C++编写，运行速度快，占用内存少

• 接口用Python暴露，使用方便，开发便捷

分布式系统

• 底层硬件同时支持CPU和GPU

• 分布式系统，支持单机运行和云上运行

部署方便

• 支持docker部署和原生包部署

• 支持多操作系统windows、 macOS、 Linux

优势分析

◆ 提供搭建私有云的全套解决方案

- ➢ 帮助用户保护数据
- ➢ 充分支持分布式系统
- ➢ 模型并行和数据并行

◆ 灵活易用且性能高

- ➢ 支持多种机器学习和优化算法，可定制
- ➢ 全面支持多机、多GPU环境，优化的通信实现使高吞吐与高性能成为可能

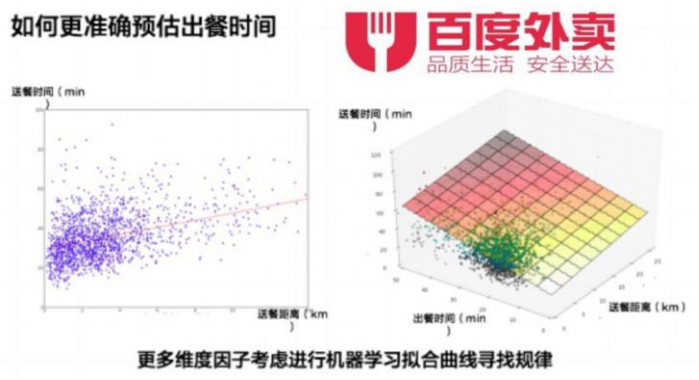
官网：http://www.paddlepaddle.org/

训练营：http://ai.baidu.com/paddlepaddle

PaddlePaddle的应用方向

PaddlePaddle已在百度30多项主要产品和服务之中发挥着巨大的作用。

外卖预估出餐时间



精准推荐用户所需信息

字符识别(OCR)

机器翻译

百度翻译手机版提供中英日韩等10多个语种的文本互译；使用OCR技术实现拍照整屏实时翻译、菜单翻译等功能，无需键盘输入，提升用户体验。

病毒和垃圾信息检测

自动驾驶

海量图像识别分类

PaddlePaddle的模型库

PaddlePaddle 为开发者提供了丰富的模型库，从而可以快速构建强大的应用，满足各种场景的需求。

计算机视觉

◆**图像分类**是根据图像的语义信息对不同类别图像进行区分，是计算机视觉中重要的基础问题。主要应用于安防领域的人脸识别和智能视频分析等，交通领域的交通场景识别等。PaddlePaddle提供了常用模型：AlexNet、VGG、GoogleNet、ResNet等，方便用户使用。

图像分类

◆**目标检测**的目标是是给定一张图像或是一个视频帧，让计算机找出其中所有目标的位置，并给出每个目标的具体类别。PaddlePaddle提供了Faster-RCNN、MobileNet-SSD、PyramidBox等模型，满足不同场景需求。

目标检测

PaddlePaddle的模型库

自然语言处理

◆**阅读理解**-DuReader模型

应用场景：语音助手、智能搜索、智能客服、智能机器人

◆**机器翻译**-Transformer模型

应用场景：机器翻译

◆**语义匹配**-DAM模型

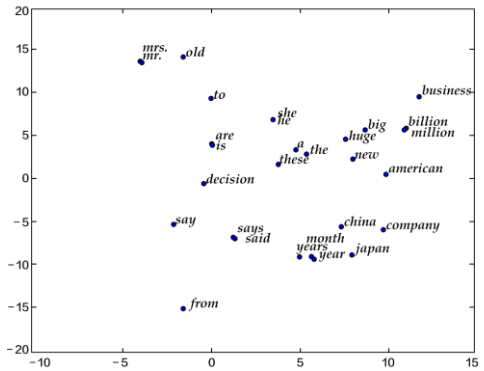
应用场景：问答系统、对话系统、智能客服等

◆**情感分析**-Senta模型

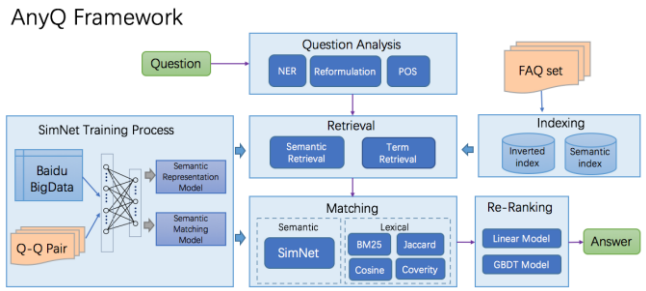
应用场景：舆情分析、热点话题分析

◆**语言模型**LSTM、GRU等

词向量



问答系统



PaddlePaddle的模型库

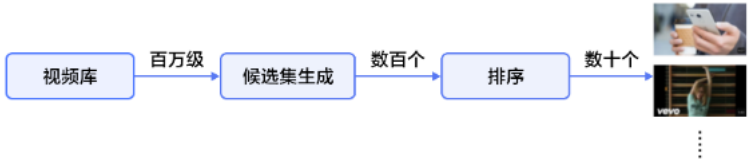
推荐

以**新闻推荐系统**为例，存在多个可以使用深度学习技术的环节，例如新闻的自动化标注，个性化新闻召回，个性化匹配与排序等。

PaddlePaddle对推荐算法的训练提供了完整的支持，并提供了多种模型配置供用户选择。如TagSpace、RRU4Rec、SSR、DeepCTR等模型

个性化推荐

个性化推荐挖掘用户的行为，将用户可能感兴趣的信息和商品推荐给用户，提高交易成功率和用户体验度



除此之外，PaddlePaddle还为开发者们提供了**语音识别、强化学习**等方面的一些模型库，可以满足各种场景需求。

PaddlePaddle的本地环境搭建

目前PaddlePaddle支持以下环境:

- Ubuntu 14.04 /16.04 /18.04
- CentOS 7 / 6
- MacOS 10.11 / 10.12 / 10.13 / 10.14
- Windows7 / 8/ 10 (专业版/企业版)

PaddlePaddle支持三种形式的安装，pip安装、docker安装、源码安装，**本教程以Windows7下pip安装**为例。

若需要docker或源码安装，请参考：

http://paddlepaddle.org/documentation/docs/zh/1.3/beginners_guide/install/install_Docker.html

PaddlePaddle的本地环境搭建

安装Python

- 下载Python3.5

  1）自行百度下载

  2）百度网盘https://pan.baidu.com/s/1t_mBGiwre7eGTio-0MRqmQ，提取码：4efy

- 安装Python3.5

勾选“Add Python3.5 to PATH”

- 安装是否成功

  cmd中执行python命令

PaddlePaddle的本地环境搭建

安装PaddlePaddle

◆在线pip安装

cmd中，执行pip install paddlepaddle,开始安装

```
C:\Users\wenxiaoxin>pip install paddlepaddle
```

```
python-4.0.0.21 paddlepaddle-1.3.0 protobuf-3.1.0 pyparsing-2.3.1 python-dateuti
l-2.8.0 pytz-2018.9 pyyaml-3.13 rarfile-3.0 recordio-0.1.7 requests-2.9.2 scipy-
1.2.1 singledispatch-3.4.0.3 six-1.12.0
You are using pip version 9.0.1, however version 19.0.3 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' comm
and.

C:\Users\wenxiaoxin>
```

◆升级pip版本

如果提示pip版本过低，按照提示，执行python -m pip install --upgrade pip，升级pip

PaddlePaddle的本地环境搭建

安装PyCharm

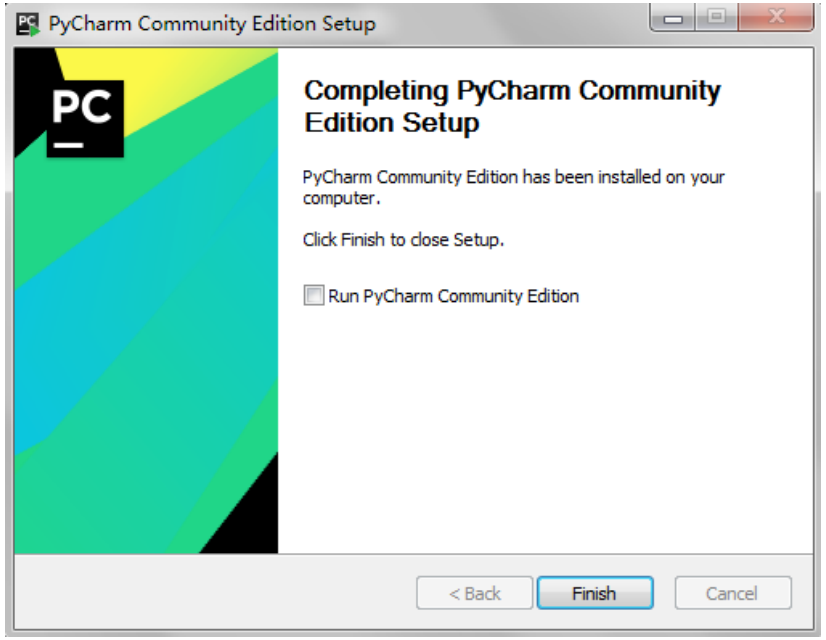
◆下载PyCharm

1）自行百度下载

2）百度网盘https://pan.baidu.com/s/1t_mBGiwre7eGTio-0MRqmQ，提取码：4efy

◆安装PyCharm

默认安装即可

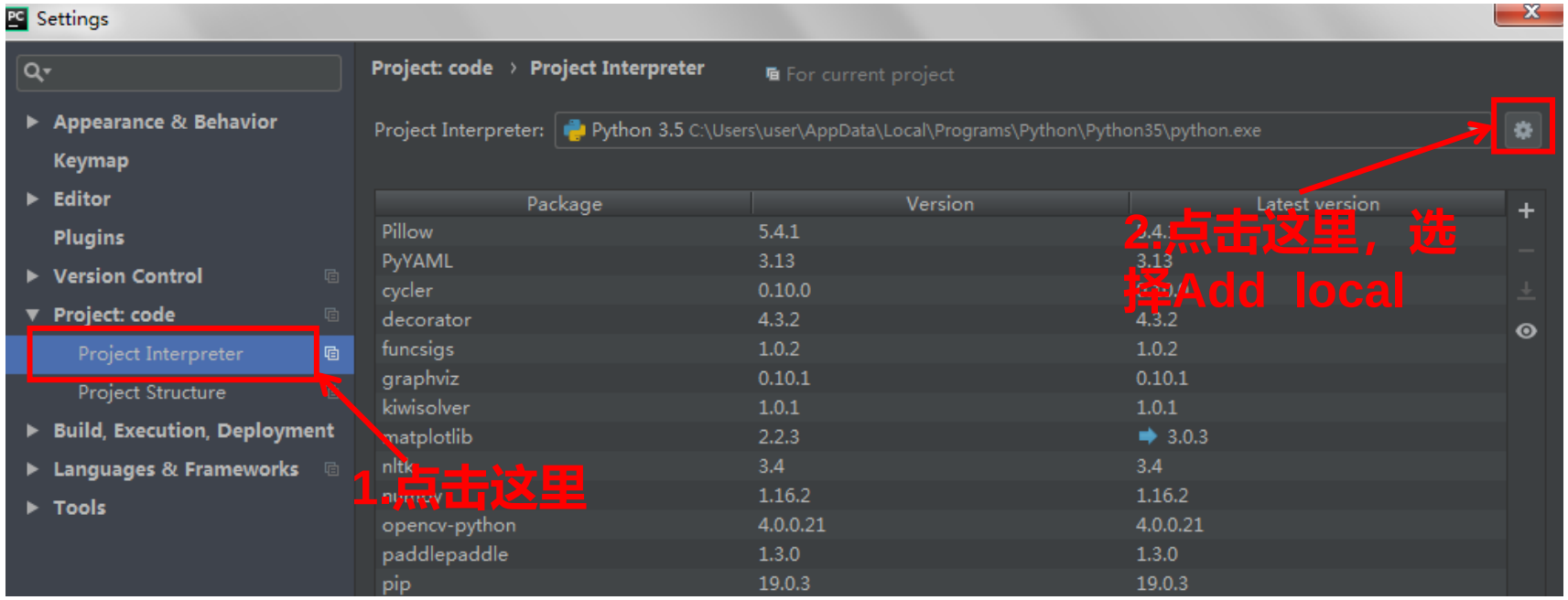


PaddlePaddle的本地环境搭建

◆配置Python解释器

1）点击file->Settings

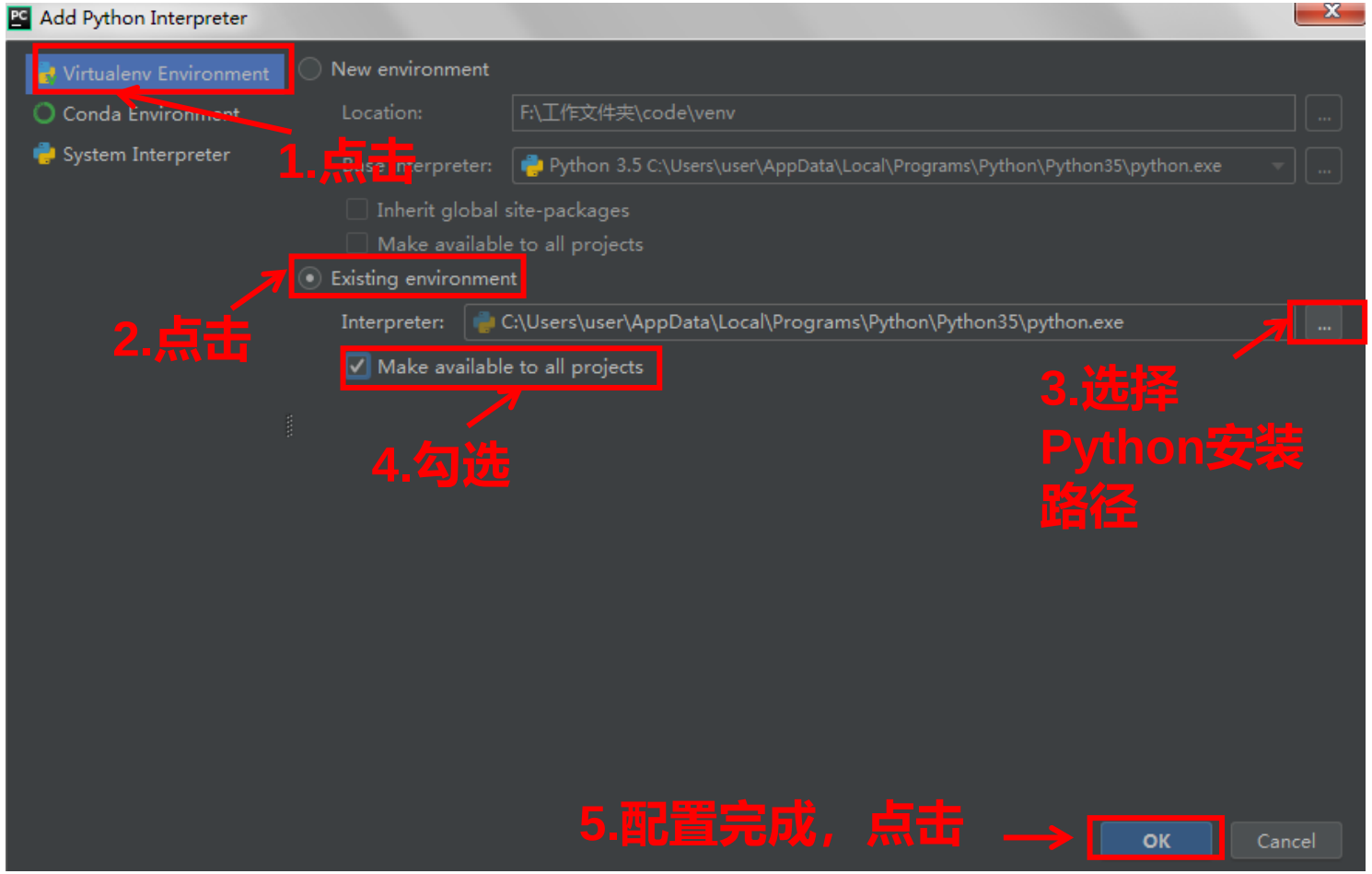
2）点击Project Interpreter,选择Add Local



PaddlePaddle的本地环境搭建

◆配置Python解释器

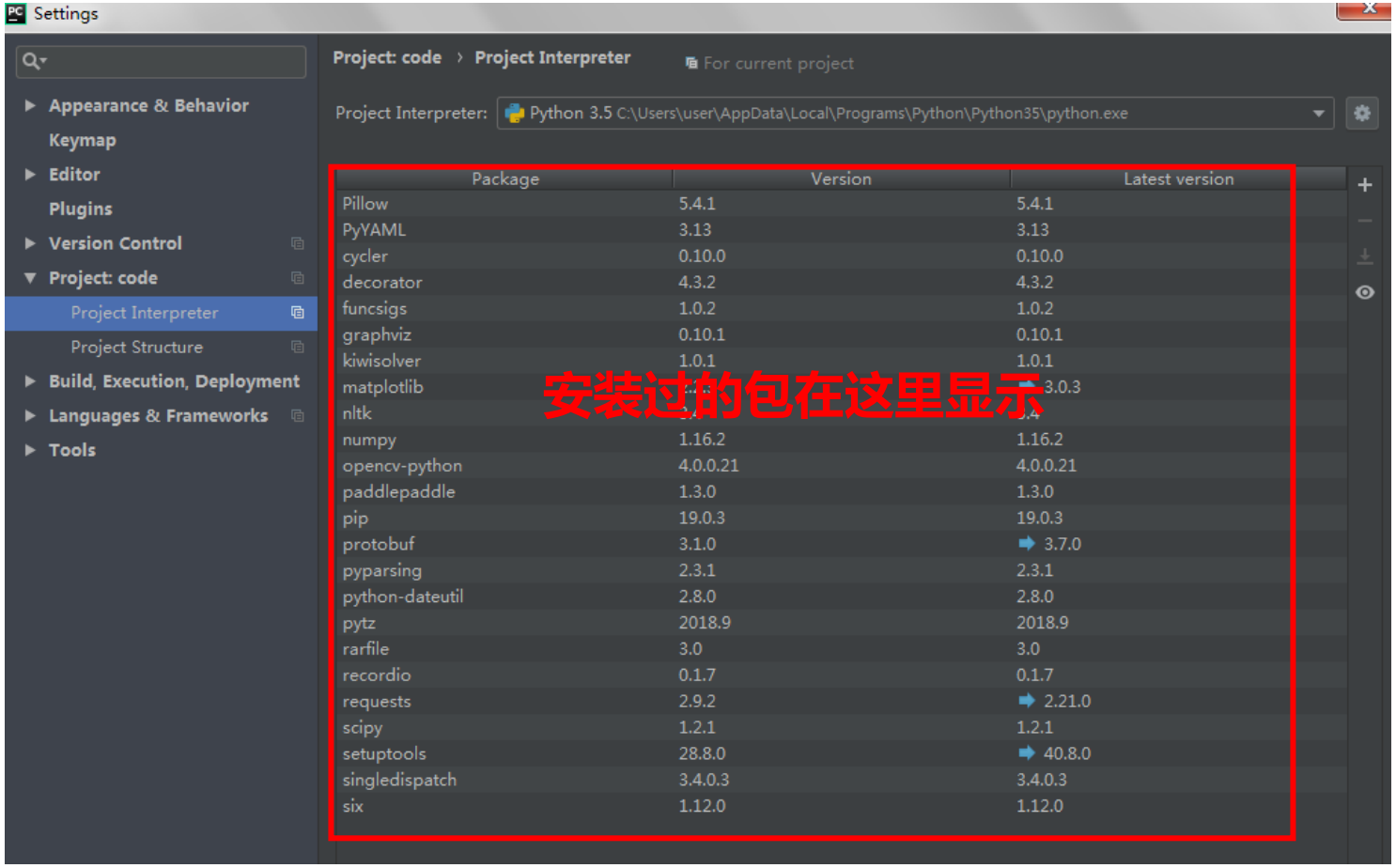
3）点击Vitualenv Environment->Existing environment，选择Python安装路径，勾选“Make available to all projects”，配置完成。



PaddlePaddle的本地环境搭建

◆配置Python解释器

4）加载完成后，会显示出Python安装过的所有包，证明PyCharm调试已经成功了。



PaddlePaddle的本地环境搭建

test.py

双击我！！

◆代码测试

将测试代码复制到PyCharm中，点击运行，出现下图左侧所示信息，则运行成功

```
Pass:0, Batch:0, Cost:2.70508, Accuracy:0.06250
Pass:0, Batch:100, Cost:0.38185, Accuracy:0.86719
Pass:0, Batch:200, Cost:0.23436, Accuracy:0.89844
Pass:0, Batch:300, Cost:0.44820, Accuracy:0.84375
Pass:0, Batch:400, Cost:0.26247, Accuracy:0.90625
Test:0, Cost:0.21892, Accuracy:0.93305
save models to /home/aistudio/data/hand.inference.model
Pass:1, Batch:0, Cost:0.27801, Accuracy:0.91406
Pass:1, Batch:100, Cost:0.28942, Accuracy:0.92188
Pass:1, Batch:200, Cost:0.16124, Accuracy:0.95312
Pass:1, Batch:300, Cost:0.21073, Accuracy:0.92188
Pass:1, Batch:400, Cost:0.22340, Accuracy:0.90625
Test:1, Cost:0.14514, Accuracy:0.95342
save models to /home/aistudio/data/hand.inference.model
```

```
Run: hand_write
W:\pycharm\venv\Scripts\python.exe W:/pycharm/paddlepaddle/hand_write.py
Traceback (most recent call last):
  File "W:/pycharm/paddlepaddle/hand_write.py", line 2, in <module>
    import numpy as np
ImportError: No module named 'numpy'

Process finished with exit code 1
```

注：若出现上图右侧所示信息，则是由于缺少Numpy安装包引起的，cmd中执行pip install numpy命令即可。

百度AIStudio简介

百度AI Studio一站式开发平台：囊括了AI教程、代码环境、算法算力、数据集，并提供免费的在线云计算的一体化编程环境。官网地址： http://aistudio.baidu.com/

实践平台！

AI Studio特性

云端集成	简单易用	运行高效	资源免费
计算资源	项目管理	资源配置	视频教程
比赛平台	项目共享	集群支撑	空间资源
样例模型	数据集成	多项目并行	计算资源
视频教程	无需安装	P40 GPU	代码管理

百度AI Studio平台已经为使用者预置了Python语言环境, 以及百度PaddlePaddle深度学习开发框架. 同时用户可以在其中自行加载Scikit-Learn等机器学习库.

百度AIStudio项目大厅

教程中心

- 新手指引　秒懂强大的在线编程平台
- AI教程　帮您轻松掌握AI开发
- 新闻中心　平台功能迭代和赛事迭代

项目列表

- 样例工程　经典应用场景
- 共享项目　项目共享功能，相互学习
- 个人项目　开发者项目列表

百度AIStudio创建项目

- 创建你自己的项目
  1）创建项目：点击按钮创建项目
  2）项目环境：Python2.7和Python3.5
  3）预加载模块：PaddlePaddle-v0.14.0和v1.0.1
  4）资源配置：单机或远程集群
  5）项目名称/项目描述: 用来标识项目, 便于日后进行查找和管理, 创建后支持修改.

百度AIStudio创建项目

- 创建你自己的项目
  - ✓如果项目涉及到数据集, 可以考虑直接使用系统预置的数据集, 点击添加数据集按钮
  - 注意：每个项目最多可以引入两个数据集, 如无 合适的数据集, 用户也可以自行上传创建新数据集.
  - ✓点击“创建”，“查看”，可进入项目详情页

百度AIStudio运行项目

◆ 项目详情页

✓进入项目详情页

✓点击进入，项目环境初始化

百度AIStudio编辑代码

◆ 代码编辑页面

百度AIStudio编辑代码

◆ 新建块

✓选择块类型：Code或Markdown,分别插入代码块或文字块

新增cell类型: Code 新增cell

✓代码块，输入代码，点击运行，在下方输出运行结果

```
1 print("Welcome to AIStudio")
```

运行输出

```
Welcome to AIStudio
```

✓文字块，支持Markdown格式, 点击【预览】则出现渲染后的效果

预览 删除 切换cell类型

B I H

Welcome to AIStudio

编辑 删除 切换cell类型

Welcome to AIStudio

百度AIStudio

◆ 命令、编辑模式

内容编辑区由基本的块(cell)组成.

绿色代表块内容可编辑状态-编辑模式（比如输入代码），

蓝色代表块可操作状态-命令模式（比如删除块，必须回到蓝色）

✓编辑模式

```
In [1]:   ▷ 运行   ⇧ 上移   ⇩ 下移                                  删除
          1  print('welcome to AI Studio')
Output    welcome to AI Studio
```

✓命令模式

```
In [1]:   ▷ 运行   ⇧ 上移   ⇩ 下移                                  删除
          1  print('welcome to AI Studio')
Output    welcome to AI Studio
```

百度AIStudio Fork项目

如果不熟悉操作, 则可以直接Fork百度AI学习项目, 或者其他开发者共享的项目, 加快学习速度.

百度AIStudio数据集

数据集列表

- 公开数据集
- 收藏数据集
- 个人数据集

百度AIStudio数据集

◆ 创建数据集

创建数据集

* 数据集名称 鸢尾花数据集

* 数据集简介 每一行数据由 4 个特征值及一个目标值组成。
4 个特征值分别为：萼片长度、萼片宽度、花瓣长度、花瓣宽度
目标值为三种不同类别的鸢尾花，分别为：Iris Setosa，Iris Versicolour，Iris Virginica。

* 上传数据集文件 iris.data × 最多输入50个字

上传文件

最多上传10个文件，单个文件不超过10GB

下一步 取 消

6 Pytorch

Pytorch的起源

- 2017年1月，Facebook人工智能研究院（FAIR）团队在Github上开源了Pytorch，并迅速占据Github热度榜榜首。
- Pytorch的历史可以追溯到2002年就诞生的Torch。Torch使用了一种不是很大众的语言Lua作为接口。
- 2017年，Torch的幕后团队对Tensor之上的所有模块进行了重构，推出了Pytorch。

PyTorch

简洁直观的设计理念

- PyTorch 的设计思路是线性、直观且易于使用的。当你的代码出现 Bug 的时候，不会因为错误的指向或者异步和不透明的引擎浪费太多的时间。
- PyTorch 的代码相对于Tensorflow 而言，更加简洁直观，同时对于Tensorflow高度工业化的很难看懂的底层代码，PyTorch 的源代码就要友好得多，更容易看懂，更加吸引初学者学习。

使用动态计算图

- Tensorflow 属于命令式的编程语言，而且是静态的，首先必须构建一个神经网络，然后一次又一次使用同样的结构，如果想要改变网络的结构，就必须从头开始。
- 而PyTorch通过一种反向自动求导的技术，可以让你零延迟地任意改变神经网络的行为，能够为你奇思妙想的实现获得最高的速度和最佳的灵活性，这也是PyTorch 对比Tensorflow 最大的优势。

感谢！